



CHRIST
(DEEMED TO BE UNIVERSITY)
DELHI - NCR, INDIA

NLDL Programming

MCA-575

Assignment – 05

BY

HIMANSHU HEDA (24225013)

SUBMITTED TO

Dr. Manjula Shannhog

SCHOOL OF SCIENCES

2025-26

```
# 1. Import Required Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
# 2. Load the Dataset
df = pd.read_csv('./Dataset/Admission_Predict.csv')
```

```
# 3. Data Exploration
print("First 5 rows:\n", df.head())
print("\nDataset shape:", df.shape)
print("\nInfo:")
df.info()
print("\nMissing values:\n", df.isnull().sum())
print("\nStatistical summary:\n", df.describe())
```

First 5 rows:

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	\
0	1	337	118	4	4.5	4.5	9.65	
1	2	324	107	4	4.0	4.5	8.87	
2	3	316	104	3	3.0	3.5	8.00	
3	4	322	110	3	3.5	2.5	8.67	
4	5	314	103	2	2.0	3.0	8.21	

	Research	Chance of Admit
0	1	0.92
1	1	0.76
2	1	0.72
3	1	0.80
4	0	0.65

Dataset shape: (400, 9)

Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 400 entries, 0 to 399

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
---	--------	----------------	-------

--- -----

```

0  Serial No.      400 non-null  int64
1  GRE Score      400 non-null  int64
2  TOEFL Score    400 non-null  int64
3  University Rating 400 non-null  int64
4  SOP            400 non-null  float64
5  LOR            400 non-null  float64
6  CGPA           400 non-null  float64
7  Research       400 non-null  int64
8  Chance of Admit 400 non-null  float64
dtypes: float64(4), int64(5)
memory usage: 28.3 KB

```

Missing values:

```

Serial No.      0
GRE Score      0
TOEFL Score     0
University Rating 0
SOP            0
LOR            0
CGPA           0
Research       0
Chance of Admit 0
dtype: int64

```

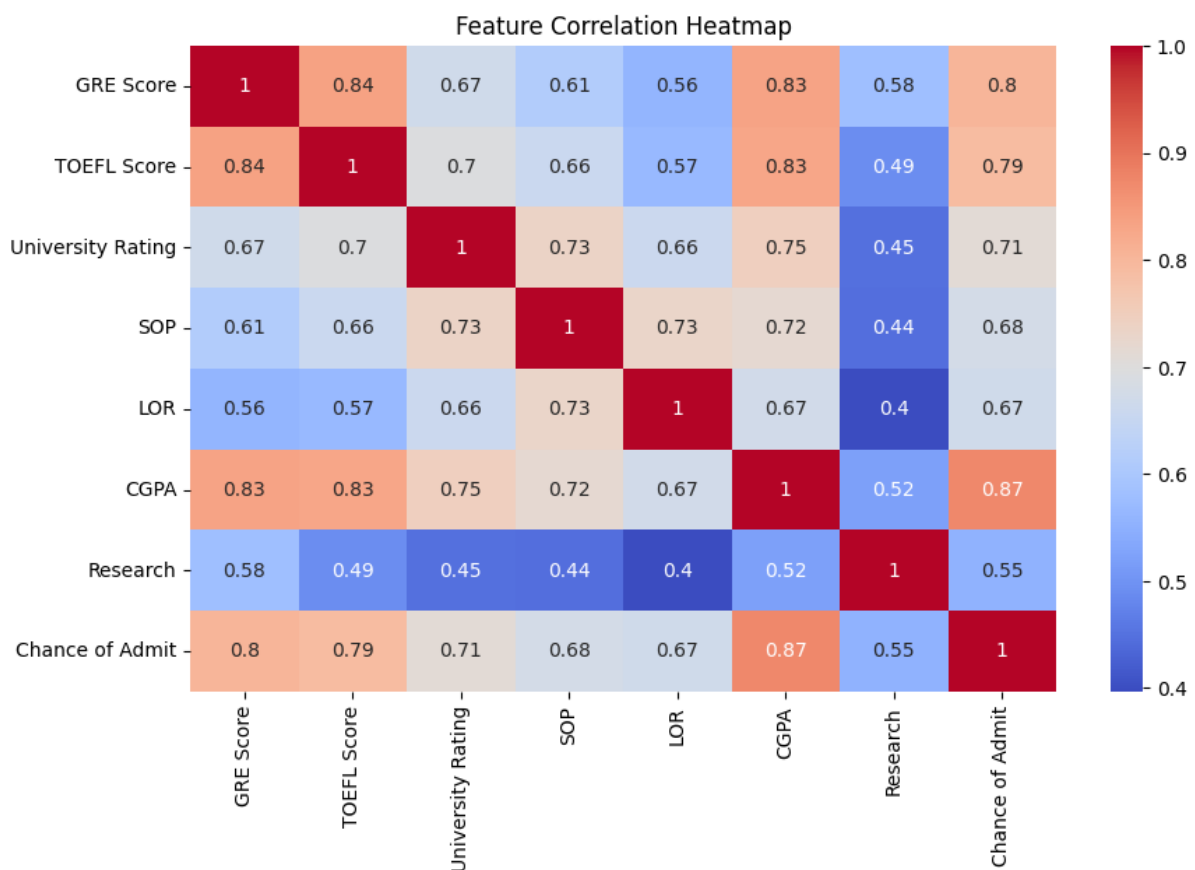
Statistical summary:

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP \
count	400.000000	400.000000	400.000000	400.000000	400.000000
mean	200.500000	316.807500	107.410000	3.087500	3.400000
std	115.614301	11.473646	6.069514	1.143728	1.006869
min	1.000000	290.000000	92.000000	1.000000	1.000000
25%	100.750000	308.000000	103.000000	2.000000	2.500000
50%	200.500000	317.000000	107.000000	3.000000	3.500000
75%	300.250000	325.000000	112.000000	4.000000	4.000000
max	400.000000	340.000000	120.000000	5.000000	5.000000

	LOR	CGPA	Research	Chance of Admit
count	400.000000	400.000000	400.000000	400.000000
mean	3.452500	8.598925	0.547500	0.724350
std	0.898478	0.596317	0.498362	0.142609
min	1.000000	6.800000	0.000000	0.340000
25%	3.000000	8.170000	0.000000	0.640000
50%	3.500000	8.610000	1.000000	0.730000
75%	4.000000	9.062500	1.000000	0.830000
max	5.000000	9.920000	1.000000	0.970000

```
# 4. Data Cleaning (if needed)
# Remove 'Serial No.' if present
if 'Serial No.' in df.columns:
    df = df.drop('Serial No.', axis=1)
```

```
# 5. Visualize Feature Distributions
plt.figure(figsize=(10, 6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Feature Correlation Heatmap')
plt.show()
```



```
# 6. Prepare Features and Target
target_col = [col for col in df.columns if 'Chance' in col][0]
X = df.drop([target_col], axis=1)
y = df[target_col]
```

```
# 7. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
# 8. Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
df.shape[1]
```

9. Build the ANN Regression Model

```
model = Sequential()  
model.add(Dense(32, activation='relu', input_dim=X_train_scaled.shape[1]))  
model.add(Dense(16, activation='relu'))  
model.add(Dense(8, activation='relu'))  
model.add(Dense(1)) # Regression output  
model.summary()
```

```
c:\Users\himan\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\dense.py:92: UserWarning: Do not pass an 'input_shape'/'input_dim' argument  
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
dense_17 (Dense)	(None, 32)	256
dense_18 (Dense)	(None, 16)	528
dense_19 (Dense)	(None, 8)	136
dense_20 (Dense)	(None, 1)	8

Total params: 928 (3.63 KB)

Trainable params: 928 (3.63 KB)

Non-trainable params: 0 (0.00 B)

10. Compile the Model

```
model.compile(optimizer='adam', loss='mean_squared_error')
```

11. Train the Model

```
history = model.fit(X_train_scaled, y_train, epochs=100, validation_split=0.2,  
verbose=1)
```

Epoch 1/100

1/8 1m8/8s 0m 32m ————— 0m 37m 0m

1m1s 0m 21ms/step - loss: 0.7867 - val_loss: 0.5374

Epoch 2/100

1/8 1m8/8s 0m 32m ————— 0m 37m 0m

1m0s 0m 7ms/step - loss: 0.3972 - val_loss: 0.2602

Epoch 3/100

1/8 1m8/8s 0m 32m ————— 0m 37m 0m

1m0s 0m 15ms/step - loss: 0.1999 - val_loss: 0.1317

Epoch 4/100

1/8 1m8/8s 0m 32m ————— 0m 37m 0m

1m0s 0m 30ms/step - loss: 0.1121 - val_loss: 0.0855

Epoch 5/100

1/8 1m8/8s 0m 32m ————— 0m 37m 0m

1m0s 0m 18ms/step - loss: 0.0851 - val_loss: 0.0731

Epoch 6/100

□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 10ms/step - loss: 0.0733 - val_loss: 0.0658
Epoch 7/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 8ms/step - loss: 0.0621 - val_loss: 0.0567
Epoch 8/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 8ms/step - loss: 0.0523 - val_loss: 0.0499
Epoch 9/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 9ms/step - loss: 0.0458 - val_loss: 0.0468
Epoch 10/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 11ms/step - loss: 0.0400 - val_loss: 0.0445
Epoch 11/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 20ms/step - loss: 0.0352 - val_loss: 0.0428
Epoch 12/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 43ms/step - loss: 0.0318 - val_loss: 0.0412
Epoch 13/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 21ms/step - loss: 0.0291 - val_loss: 0.0400
Epoch 14/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 13ms/step - loss: 0.0265 - val_loss: 0.0379
Epoch 15/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 40ms/step - loss: 0.0243 - val_loss: 0.0361
Epoch 16/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 23ms/step - loss: 0.0224 - val_loss: 0.0349
Epoch 17/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 26ms/step - loss: 0.0207 - val_loss: 0.0334
Epoch 18/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 36ms/step - loss: 0.0193 - val_loss: 0.0319
Epoch 19/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 21ms/step - loss: 0.0179 - val_loss: 0.0306
Epoch 20/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 27ms/step - loss: 0.0166 - val_loss: 0.0294
Epoch 21/100

□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 13ms/step - loss: 0.0156 - val_loss: 0.0281
Epoch 22/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 24ms/step - loss: 0.0147 - val_loss: 0.0270
Epoch 23/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 23ms/step - loss: 0.0138 - val_loss: 0.0259
Epoch 24/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 8ms/step - loss: 0.0131 - val_loss: 0.0249
Epoch 25/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 7ms/step - loss: 0.0124 - val_loss: 0.0241
Epoch 26/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 11ms/step - loss: 0.0118 - val_loss: 0.0231
Epoch 27/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 34ms/step - loss: 0.0111 - val_loss: 0.0222
Epoch 28/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 26ms/step - loss: 0.0106 - val_loss: 0.0214
Epoch 29/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 28ms/step - loss: 0.0101 - val_loss: 0.0208
Epoch 30/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 24ms/step - loss: 0.0097 - val_loss: 0.0203
Epoch 31/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 17ms/step - loss: 0.0092 - val_loss: 0.0196
Epoch 32/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 24ms/step - loss: 0.0088 - val_loss: 0.0189
Epoch 33/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 25ms/step - loss: 0.0085 - val_loss: 0.0184
Epoch 34/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 24ms/step - loss: 0.0081 - val_loss: 0.0177
Epoch 35/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 33ms/step - loss: 0.0078 - val_loss: 0.0171
Epoch 36/100

□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 11ms/step - loss: 0.0076 - val_loss: 0.0166
Epoch 37/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 6ms/step - loss: 0.0073 - val_loss: 0.0161
Epoch 38/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 14ms/step - loss: 0.0070 - val_loss: 0.0157
Epoch 39/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 16ms/step - loss: 0.0068 - val_loss: 0.0153
Epoch 40/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 24ms/step - loss: 0.0066 - val_loss: 0.0149
Epoch 41/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 17ms/step - loss: 0.0064 - val_loss: 0.0146
Epoch 42/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 6ms/step - loss: 0.0063 - val_loss: 0.0143
Epoch 43/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 6ms/step - loss: 0.0061 - val_loss: 0.0140
Epoch 44/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 6ms/step - loss: 0.0059 - val_loss: 0.0137
Epoch 45/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 10ms/step - loss: 0.0058 - val_loss: 0.0134
Epoch 46/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 40ms/step - loss: 0.0056 - val_loss: 0.0131
Epoch 47/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 26ms/step - loss: 0.0055 - val_loss: 0.0128
Epoch 48/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 13ms/step - loss: 0.0054 - val_loss: 0.0126
Epoch 49/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 14ms/step - loss: 0.0052 - val_loss: 0.0123
Epoch 50/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 22ms/step - loss: 0.0051 - val_loss: 0.0120
Epoch 51/100

□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 10ms/step - loss: 0.0050 - val_loss: 0.0119
Epoch 52/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 26ms/step - loss: 0.0049 - val_loss: 0.0116
Epoch 53/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 21ms/step - loss: 0.0049 - val_loss: 0.0114
Epoch 54/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 25ms/step - loss: 0.0047 - val_loss: 0.0112
Epoch 55/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 24ms/step - loss: 0.0047 - val_loss: 0.0109
Epoch 56/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 17ms/step - loss: 0.0046 - val_loss: 0.0108
Epoch 57/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 13ms/step - loss: 0.0045 - val_loss: 0.0106
Epoch 58/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 17ms/step - loss: 0.0044 - val_loss: 0.0105
Epoch 59/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 18ms/step - loss: 0.0044 - val_loss: 0.0103
Epoch 60/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 22ms/step - loss: 0.0043 - val_loss: 0.0102
Epoch 61/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 17ms/step - loss: 0.0042 - val_loss: 0.0101
Epoch 62/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 30ms/step - loss: 0.0042 - val_loss: 0.0100
Epoch 63/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 19ms/step - loss: 0.0041 - val_loss: 0.0098
Epoch 64/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 15ms/step - loss: 0.0041 - val_loss: 0.0098
Epoch 65/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 19ms/step - loss: 0.0040 - val_loss: 0.0096
Epoch 66/100

□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 30ms/step - loss: 0.0039 - val_loss: 0.0096
Epoch 67/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 16ms/step - loss: 0.0039 - val_loss: 0.0094
Epoch 68/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 12ms/step - loss: 0.0038 - val_loss: 0.0094
Epoch 69/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 15ms/step - loss: 0.0038 - val_loss: 0.0093
Epoch 70/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 12ms/step - loss: 0.0037 - val_loss: 0.0093
Epoch 71/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 8ms/step - loss: 0.0037 - val_loss: 0.0092
Epoch 72/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 8ms/step - loss: 0.0036 - val_loss: 0.0091
Epoch 73/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 9ms/step - loss: 0.0036 - val_loss: 0.0090
Epoch 74/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 11ms/step - loss: 0.0036 - val_loss: 0.0090
Epoch 75/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 7ms/step - loss: 0.0035 - val_loss: 0.0089
Epoch 76/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 8ms/step - loss: 0.0035 - val_loss: 0.0089
Epoch 77/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 14ms/step - loss: 0.0035 - val_loss: 0.0089
Epoch 78/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 11ms/step - loss: 0.0035 - val_loss: 0.0088
Epoch 79/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 9ms/step - loss: 0.0035 - val_loss: 0.0087
Epoch 80/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 8ms/step - loss: 0.0034 - val_loss: 0.0087
Epoch 81/100

□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 11ms/step - loss: 0.0033 - val_loss: 0.0087
Epoch 82/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 13ms/step - loss: 0.0034 - val_loss: 0.0087
Epoch 83/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 33ms/step - loss: 0.0034 - val_loss: 0.0087
Epoch 84/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 35ms/step - loss: 0.0033 - val_loss: 0.0086
Epoch 85/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 20ms/step - loss: 0.0033 - val_loss: 0.0086
Epoch 86/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 20ms/step - loss: 0.0032 - val_loss: 0.0086
Epoch 87/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 17ms/step - loss: 0.0032 - val_loss: 0.0085
Epoch 88/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 9ms/step - loss: 0.0031 - val_loss: 0.0085
Epoch 89/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 28ms/step - loss: 0.0031 - val_loss: 0.0085
Epoch 90/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 7ms/step - loss: 0.0031 - val_loss: 0.0085
Epoch 91/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 17ms/step - loss: 0.0031 - val_loss: 0.0084
Epoch 92/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 30ms/step - loss: 0.0030 - val_loss: 0.0084
Epoch 93/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 35ms/step - loss: 0.0030 - val_loss: 0.0084
Epoch 94/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 21ms/step - loss: 0.0030 - val_loss: 0.0084
Epoch 95/100
□[1m8/8□[0m □[32m—————□[0m□[37m□[0m
□[1m0s□[0m 19ms/step - loss: 0.0030 - val_loss: 0.0084
Epoch 96/100

```

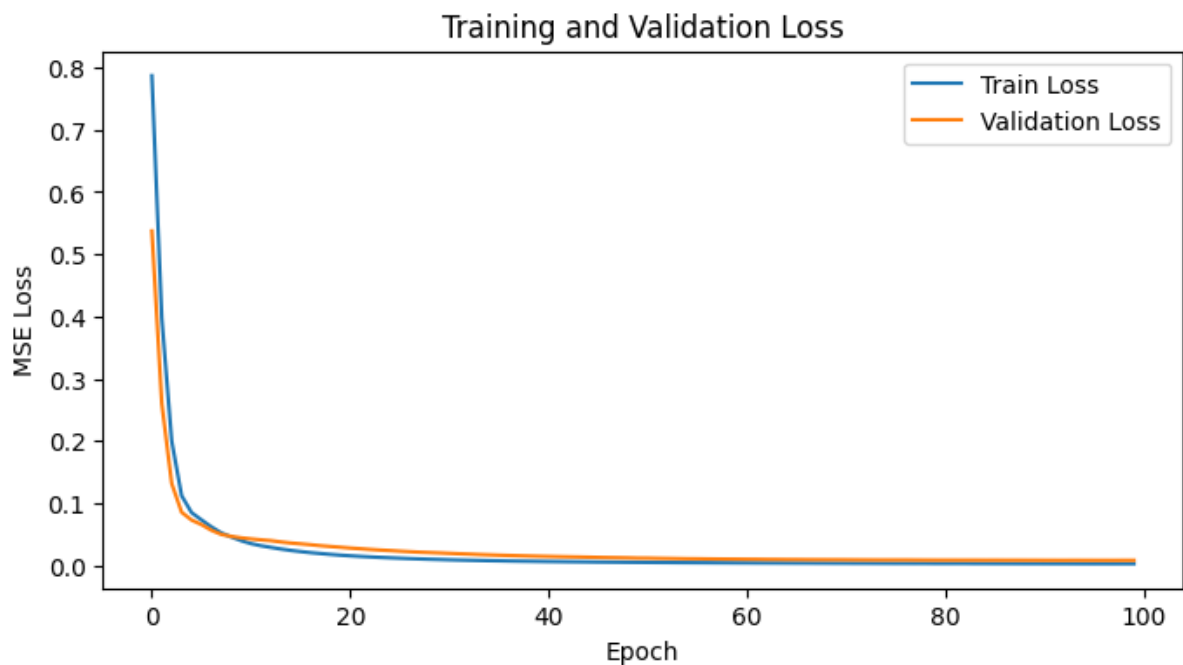
[1m8/8[0m [32m[0m[37m[0m
[1m0s[0m 35ms/step - loss: 0.0029 - val_loss: 0.0083
Epoch 97/100
[1m8/8[0m [32m[0m[37m[0m
[1m0s[0m 23ms/step - loss: 0.0030 - val_loss: 0.0083
Epoch 98/100
[1m8/8[0m [32m[0m[37m[0m
[1m0s[0m 9ms/step - loss: 0.0029 - val_loss: 0.0083
Epoch 99/100
[1m8/8[0m [32m[0m[37m[0m
[1m0s[0m 20ms/step - loss: 0.0029 - val_loss: 0.0084
Epoch 100/100
[1m8/8[0m [32m[0m[37m[0m
[1m0s[0m 37ms/step - loss: 0.0029 - val_loss: 0.0083

```

```

# 12. Plot Training History
plt.figure(figsize=(8, 4))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.show()

```



```

# 13. Evaluate the Model
loss = model.evaluate(X_test_scaled, y_test)
print('Test Loss (MSE):', loss)

```

1m3/3 0m 32m 0m 37m 0m
1m0s 0m 6ms/step - loss: 0.0080
Test Loss (MSE): 0.008010020479559898

```
# 14. Predict and Calculate Metrics
y_pred = model.predict(X_test_scaled)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print('Test Mean Squared Error (MSE):', mse)
print('Test R^2 Score (Accuracy):', r2)
```

1m3/3 0m 32m 0m 37m 0m
1m0s 0m 17ms/step
Test Mean Squared Error (MSE): 0.008010020563297669
Test R^2 Score (Accuracy): 0.6898149289184933

```
# 15. Plot Actual vs Predicted
plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_pred, alpha=0.7)
plt.xlabel('Actual Chance of Admit')
plt.ylabel('Predicted Chance of Admit')
plt.title('Actual vs Predicted')
plt.plot([0, 1], [0, 1], 'r--')
plt.show()
```

